ERPA basic tutorial

This notebook demonstrates the main ERPA workflow. The appendix introduces the functional data analysis methods.

Imports

In [1]:
```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams["svg.fonttype"] = "none" # include fonts as fonts in save
import seaborn as sns
sns.set_theme(style="ticks", context="notebook") # style preferences
from IPython.display import display
from scipy.stats import bootstrap

from erpa.core.session import load_session, prepare_figure_trials
from erpa.fda.functional import run_fpca
from erpa.fda.registration import register_all
from erpa.plotting.corr_clustermap import corr_clustermap
from erpa.plotting.trajectories import plot_trajectories, plot_trial,
from erpa.spatiotemporal.curves import event_locked_curves, movement_c
from erpa.spatiotemporal.measures import build_measure_table
```

A design decision was to use accessible colors throughout the package. We use the Okabe-Ito palette. Information about the palette is here:

https://jfly.uni-koeln.de/color/

In [2]:
```
OKABE_ITO = {
    "orange": "#E69F00",
    "sky_blue": "#56B4E9",
    "bluish_green": "#009E73",
    "blue": "#0072B2",
    "vermillion": "#D55E00",
    "reddish_purple": "#CC79A7",
    "black": "#000000",
}
```

Paths and settings

The dataset for this tutorial was synthetically generated. It is based on behavioral,

kinematic, and geometric measures from a cohort of rats perfoming a 2AFC task in the Laubach Lab.

In [3]:
```
DATA_DIR = Path("data")
OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

SLEAP_FILE = DATA_DIR / "pose.h5"
BEHAVIOR_CSV = DATA_DIR / "behavior.csv"

FRAME0_TIME = 5.32 # measured from the video for this example dataset
TRIAL_OFFSET = 0
TRIAL_INDEX = 1

EVENT_PRE_S = 0.5
EVENT_POST_S = 1.0

FDA_N_POINTS = 50
FDA_N_COMPONENTS = 3
FDA_MAX_ITER = 8
```

1. Load the session

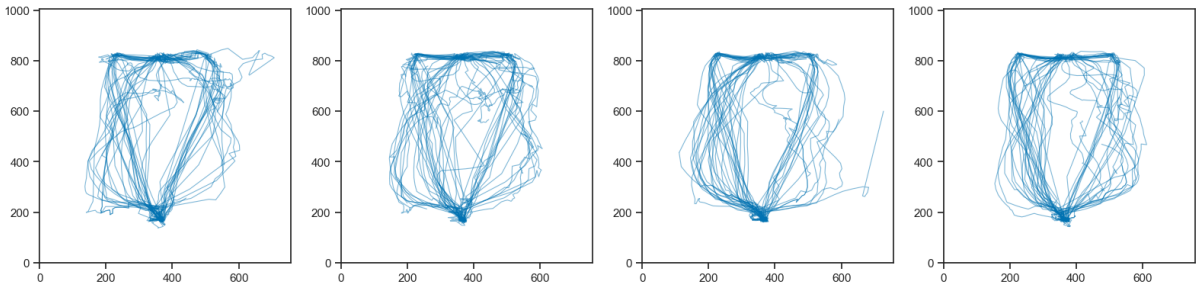
In [4]:
```
trials, behavior, series, ports, frame0_time = load_session(
    sleap_file=SLEAP_FILE,
    behavioral_csv=BEHAVIOR_CSV,
    frame0_time=FRAME0_TIME,
    trial_offset=TRIAL_OFFSET,
    pre_s=4.0,
    post_s=4.0,
)

print(f"Loaded {len(trials)} trials")
print(f"Frame-zero time: {frame0_time:.3f} s")
display(behavior.head())
```

```
load_behavior_csv dropped 26 of 239 trials with a missing outcome, nega
tive sampling, or sampling/rt above the 95th percentile.
Loaded 213 trials
Frame-zero time: 5.320 s
```

	name	date	session	sess	time	sampling	rt	cue	target	choice	re
0	JP10	2024-07-25	pbs	1.0	44.374	0.038	0.796	4.0	0.0	0.0	
1	JP10	2024-07-25	pbs	1.0	60.014	0.146	0.256	1.0	0.0	0.0	
2	JP10	2024-07-25	pbs	1.0	66.122	0.116	0.280	16.0	1.0	1.0	
3	JP10	2024-07-25	pbs	1.0	73.422	0.164	1.222	4.0	1.0	0.0	
4	JP10	2024-07-25	pbs	1.0	89.588	0.144	0.214	1.0	0.0	0.0	

2. View the trajectories

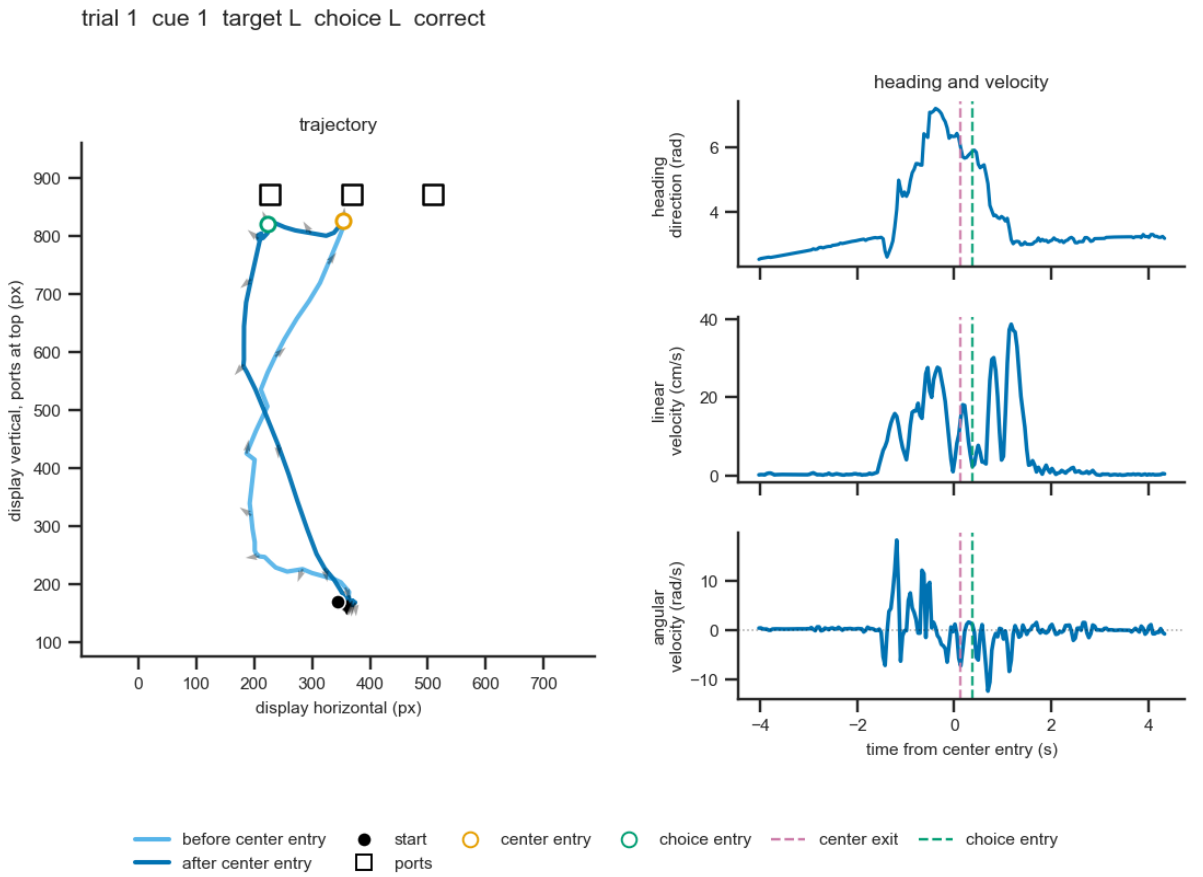
In [5]:
```
figure_trials = prepare_figure_trials(trials)

fig = plot_trajectories(
    figure_trials[:100],
    color_by=None,
    show_extrema=False,
    show_full_arena=True,
    max_trials_per_panel=25,
    orientation="paper",
)
```



3. View a single trial

In [6]:
```
fig = plot_trial(
    trials[TRIAL_INDEX],
    ports=ports,
    mark_events=(
        "center_entry",
        "choice_entry",
    ),
    figsize=(8, 6),
```

```
    dpi=140)
plt.savefig('single_trial.png', dpi=300)
plt.savefig('single_trial.svg')
```



Event-related curve helpers

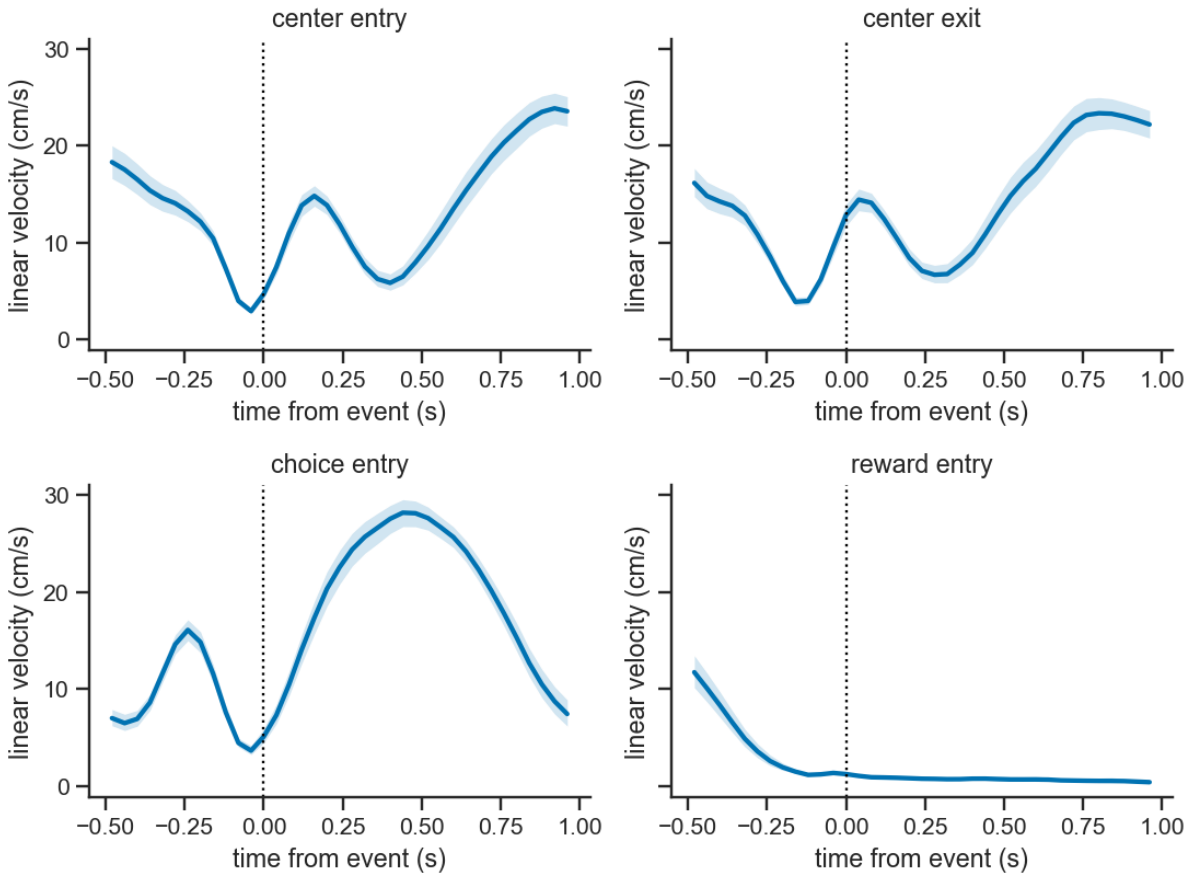
In [7]:
```
def first_column(frame, *names):
    for name in names:
        if name in frame.columns:
            return name
    raise KeyError(f"None of these columns were found: {names}")


def plot_mean_ci(ax, t, curves, mask=None, label=None, color=None):
    Y = np.asarray(curves, dtype=float)
    if mask is not None:
        Y = Y[np.asarray(mask)]

    mean = np.nanmean(Y, axis=0)
    ax.plot(t, mean, linewidth=2.2, label=label, color=color)

    if Y.shape[0] >= 3:
        result = bootstrap(
            (Y,),
            np.nanmean,
```

```
            axis=0,
            confidence_level=0.95,
            random_state=0,
        )
        ax.fill_between(
            t,
            result.confidence_interval.low,
            result.confidence_interval.high,
            alpha=0.18,
            color=color,
            linewidth=0,
        )

    ax.axvline(0, color=OKABE_ITO["black"], linestyle=":", linewidth=1
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
```

4. Linear velocity at the four main task events

In [8]:
```
EVENTS = (
    "center_entry",
    "center_exit",
    "choice_entry",
    "reward_entry",
)

event_data = {}

fig, axes = plt.subplots(2, 2, figsize=(8, 6), dpi=140, sharey=True)

for ax, event in zip(axes.flat, EVENTS):
    t, Y, meta = event_locked_curves(
        trials,
        event=event,
        key="lin_vel",
        pre_s=EVENT_PRE_S,
        post_s=EVENT_POST_S,
    )
    event_data[event] = {"time": t, "curves": Y, "meta": meta}

    plot_mean_ci(
        ax,
        t,
        Y,
        color=OKABE_ITO["blue"],
    )
    ax.set_title(event.replace("_", " "))
    ax.set_xlabel("time from event (s)")
    ax.set_ylabel("linear velocity (cm/s)")

fig.tight_layout()
```

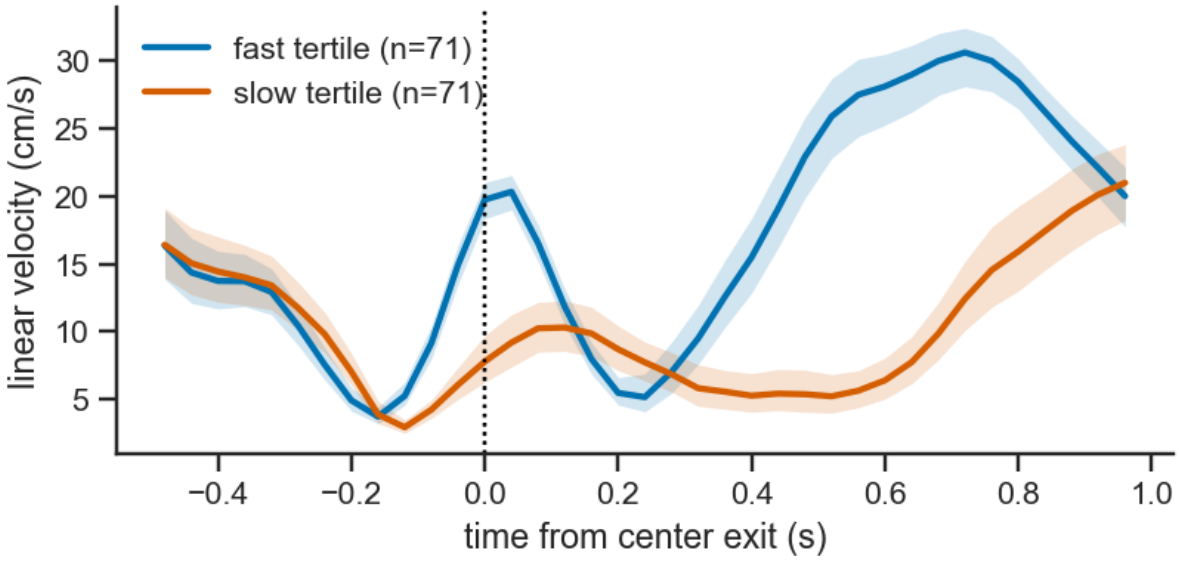
```
/Users/mark.laubach/Desktop/temp/erpa/erpa/spatiotemporal/curves.py:28
2: UserWarning: event_locked_curves skipped 2 trials whose reward_entry
window ran past the trial bounds. Rebuild span trials with larger pre_s
or post_s in load_session to make room.
  warnings.warn(
```



5. Fast and slow RT trials, defined using tertiles

In [9]:
```
center_exit = event_data["center_exit"]
rt_column = first_column(center_exit["meta"], "rt", "RT")

rt = center_exit["meta"][rt_column]
lower, upper = rt.quantile([1 / 3, 2 / 3])

fast = rt <= lower
slow = rt >= upper

fig, ax = plt.subplots(figsize=(6, 3), dpi=140)

plot_mean_ci(
    ax,
    center_exit["time"],
    center_exit["curves"],
    fast,
    label=f"fast tertile (n={fast.sum()})",
    color=OKABE_ITO["blue"],
```

```
)
plot_mean_ci(
    ax,
    center_exit["time"],
    center_exit["curves"],
    slow,
    label=f"slow tertile (n={slow.sum()})",
    color=OKABE_ITO["vermillion"],
)

ax.set_xlabel("time from center exit (s)")
ax.set_ylabel("linear velocity (cm/s)")
ax.legend(frameon=False)
fig.tight_layout()
```



6. Save the event-related curves

The compressed arrays contain one row per trial. The metadata files contain the trial identifiers and labels. ROCKET-family classifiers can use the full curves, while shapelet methods can identify discriminative subsequences. The `tsc` module in `extras/tsc` provides code to run these time-series classifiers.

In [10]:
```
for event, result in event_data.items():
    np.savez_compressed(
        OUTPUT_DIR / f"lin_vel_{event}.npz",
        time=result["time"],
        curves=result["curves"],
    )
    result["meta"].to_csv(
        OUTPUT_DIR / f"lin_vel_{event}_metadata.csv",
        index=False,
    )
```

```
X_center_exit = center_exit["curves"][:, np.newaxis, :]
print("Time-series array:", X_center_exit.shape)
```

```
Time-series array: (213, 1, 37)
```

7. Build the scalar measure table with FDA lag

In [11]:
```
measure_table, exclusions = build_measure_table(
    trials,
    ports,
    node="centroid",
    add_fda=True,
    key="lin_vel",
    n_points=FDA_N_POINTS,
    max_iter=FDA_MAX_ITER,
    return_exclusions=True,
)

print("Measure table:", measure_table.shape)
print("Scalar-only trials:", len(exclusions["scalar_only"]))
print("FDA-only trials:", len(exclusions["fda_only"]))
display(measure_table.head())
```

```
Measure table: (213, 24)
Scalar-only trials: 0
FDA-only trials: 0
```

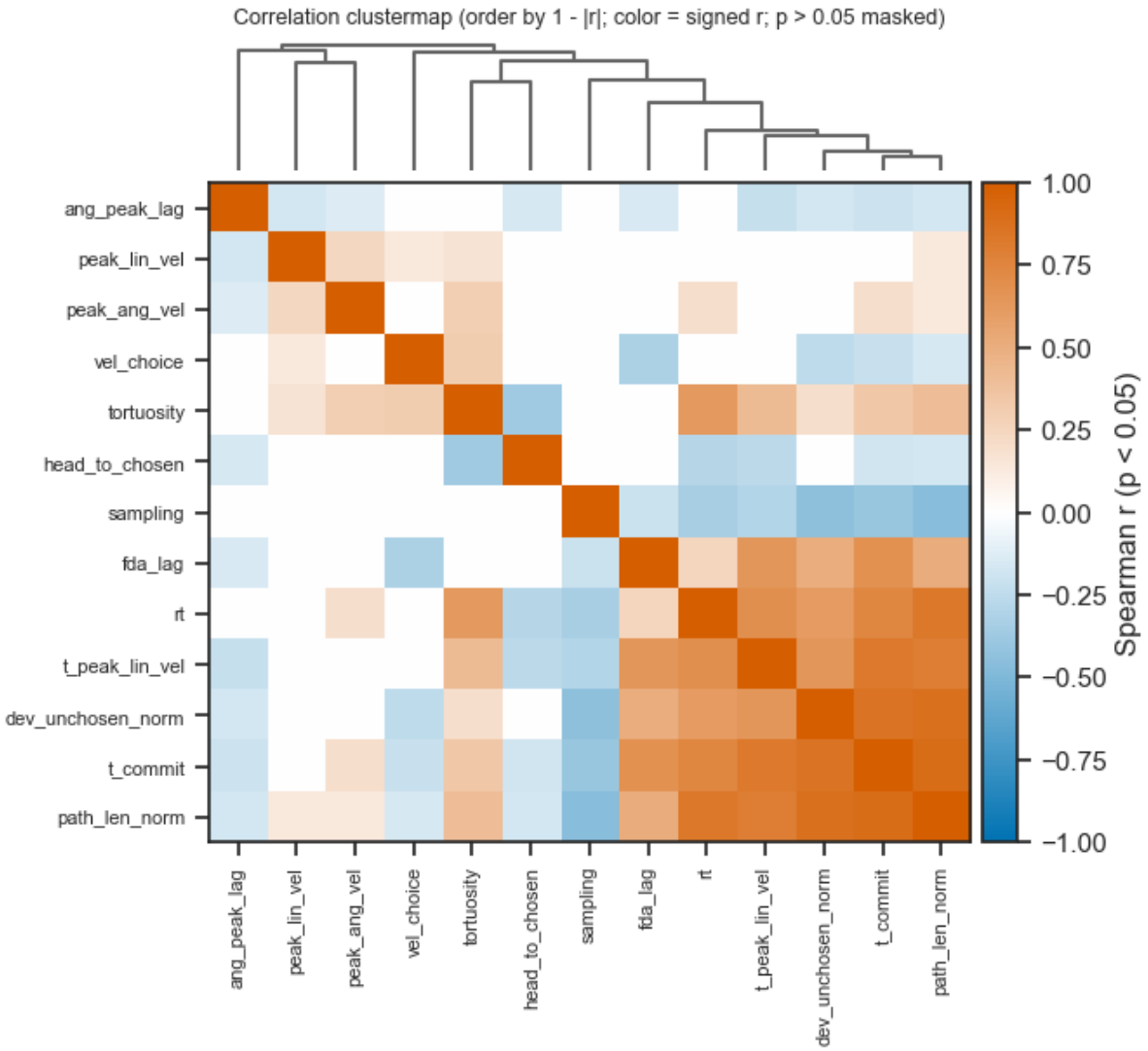
	idx	absolute_trial	target	choice	trial_type	error	cue	sampling	rt	ve
0	0	1	0	0	NaN	0	4.0	0.038	0.796	5
1	1	3	0	0	NaN	0	1.0	0.146	0.256	2
2	2	4	1	1	NaN	0	16.0	0.116	0.280	3
3	3	5	1	0	NaN	1	4.0	0.164	1.222	3
4	4	6	0	0	NaN	0	1.0	0.144	0.214	0

5 rows × 24 columns

8. Correlation clustermap

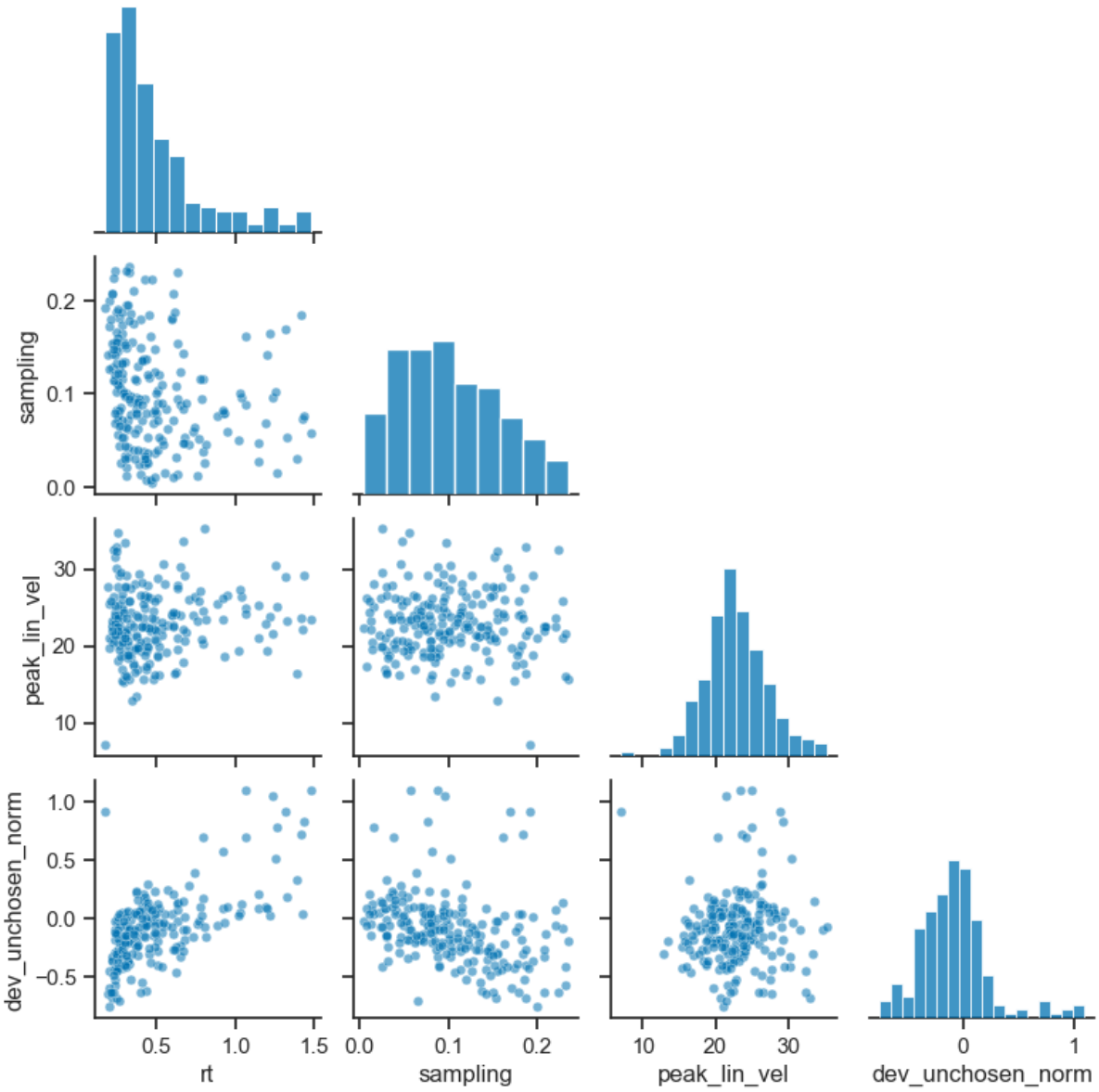
In [12]:
```
MEASURE_CANDIDATES = [
    "sampling",
    "rt",
    "peak_lin_vel",
    "peak_ang_vel",
    "t_peak_lin_vel",
    "ang_peak_lag",
```

```
    "vel_choice",
    "t_commit",
    "dev_unchosen_norm",
    "head_to_chosen",
    "tortuosity",
    "path_len_norm",
    "fda_lag",
]

measures = []
for name in MEASURE_CANDIDATES:
    if name in measure_table.columns and name not in measures:
        measures.append(name)

fig, info = corr_clustermap(
    measure_table,
    measures=measures,
    method="average",
    figsize=(6, 6),
)
```



9. Explore relationships between the measures with seaborn

In [13]:
```
rt_measure = first_column(measure_table, "rt", "RT")

pair_variables = [
    rt_measure,
    "sampling",
    "peak_lin_vel",
    "dev_unchosen_norm",
]
pair_variables = [
    name for name in pair_variables
    if name in measure_table.columns
]

plot_columns = pair_variables
plot_data = (
    measure_table[plot_columns]
    .replace([np.inf, -np.inf], np.nan)
    .dropna()
)

grid = sns.pairplot(
    plot_data,
    vars=pair_variables,
    corner=True,
    diag_kind="hist",
    plot_kws={"s": 24, "alpha": 0.55, "color": "#0072B2"},
    diag_kws={"color": "#0072B2"},  # for the diagonal distribution pl
    height=2,
)
```



10. Explore single trial trajectories based on the scalar measures

The plotting function `plot_trajectory_grid` can be used to explore trials in the session, e.g., trials with the largest and smallest values for any of the scalar measures. Here, we illustrate differences between trials with the largest and smallest deviations towards the unchosen option.

deviation_unchosen_norm

In [14]:
```
measure_table[MEASURE_CANDIDATES].loc[measure_table['dev_unchosen_norm
```

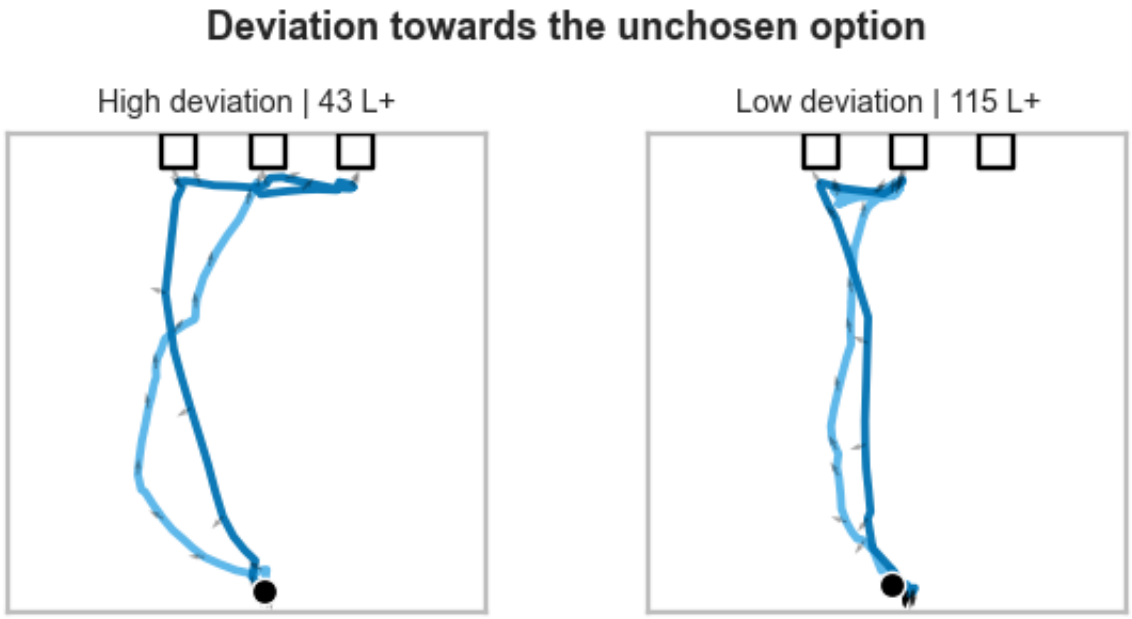
Out[14]:
```
sampling              0.088000
rt                    1.062000
peak_lin_vel         24.962896
peak_ang_vel        -24.252901
t_peak_lin_vel        0.520000
ang_peak_lag          0.040000
vel_choice            0.630203
t_commit              0.840000
dev_unchosen_norm     1.095009
head_to_chosen       -0.396962
tortuosity            2.546999
path_len_norm         3.010183
fda_lag               0.112224
Name: 43, dtype: float64
```

In [15]:
```
measure_table[MEASURE_CANDIDATES].loc[measure_table['dev_unchosen_norm
```

Out[15]:
```
sampling              0.200000
rt                    0.200000
peak_lin_vel         21.011369
peak_ang_vel        -15.126107
t_peak_lin_vel        0.000000
ang_peak_lag          0.000000
vel_choice           10.042010
t_commit              0.000000
dev_unchosen_norm    -0.758232
head_to_chosen        0.648876
tortuosity            1.015703
path_len_norm         0.201127
fda_lag              -0.119310
Name: 115, dtype: float64
```

In [16]:
```
tr_dev_unchosen_max = measure_table['idx'].loc[measure_table['dev_unch
tr_dev_unchosen_min = measure_table['idx'].loc[measure_table['dev_unch
```

In [17]:
```
# Plot trajectories for trials with the largest and smallest values fo
fig = plot_trajectory_grid(trials, ports, n_cols=2, trial_indices=[tr_

# Iterate through the axes to update the subtitles
Labels = ['High deviation', 'Low deviation']
for ind, ax in enumerate(fig.axes):
    current_title = ax.get_title()
    ax.set_title(f"{Labels[ind]} | {current_title}", fontsize=8)

fig.suptitle("Deviation towards the unchosen option", fontsize=10, fon
fig.subplots_adjust(wspace=1, top=1)

fig.tight_layout() # Re-adjust layout
plt.savefig('Trajectories_deviation.png', dpi=200)
plt.savefig('Trajectories_deviation.svg')
```



11. Save the scalar table

The table can be used for PCA, cluster analysis, tabular classification, hierarchical regression, or drift-diffusion modeling.

Examples of applying these methods to ERPA data are provided in the `extra` directory in our repository.

In [18]:
```
measure_table.to_csv(
    OUTPUT_DIR / "erpa_measure_table.csv",
    index=False,
)
```

Appendix: Functional data analysis

A1. Registration

The scalar FDA path uses the center-exit-to-choice-entry movement segment, resampled to a common normalized domain.

Registration estimates residual phase variation within that movement segment.

In [19]:
```
movement_grid, movement_velocity, movement_meta = movement_curves(
    trials,
```

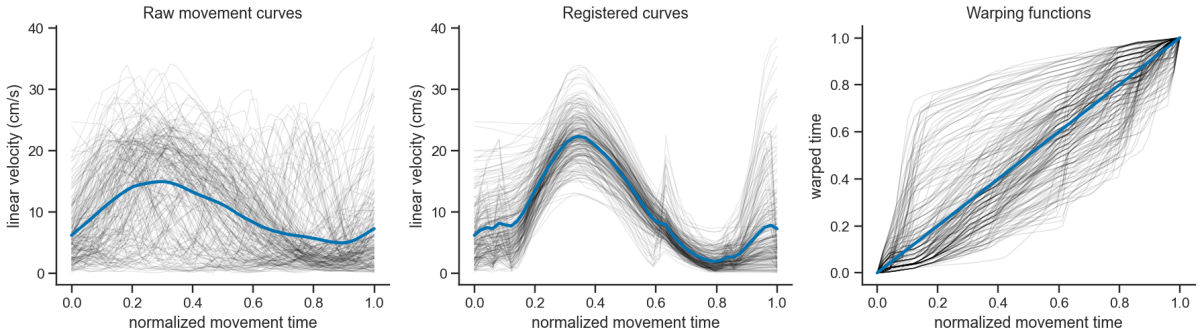
```
    key="lin_vel",
    n_points=FDA_N_POINTS,
)

registration = register_all(
    movement_grid,
    movement_velocity,
    n_comp=FDA_N_COMPONENTS,
    max_iter=FDA_MAX_ITER,
    phase_joint=False,
)

print("Raw curves:", movement_velocity.shape)
print("Registered curves:", registration["fn"].shape)
print("Warping functions:", registration["gam"].shape)
```

```
Raw curves: (213, 50)
Registered curves: (213, 50)
Warping functions: (213, 50)
```

In [20]:

```
fig, axes = plt.subplots(1, 3, figsize=(13, 3.8), dpi=140)

axes[0].plot(
    movement_grid,
    movement_velocity.T,
    color=OKABE_ITO["black"],
    alpha=0.10,
    linewidth=0.6,
)
axes[0].plot(
    movement_grid,
    np.nanmean(movement_velocity, axis=0),
    color=OKABE_ITO["blue"],
    linewidth=2.4,
)
axes[0].set_title("Raw movement curves")

axes[1].plot(
    movement_grid,
    registration["fn"].T,
    color=OKABE_ITO["black"],
    alpha=0.10,
    linewidth=0.6,
)
axes[1].plot(
    movement_grid,
    np.nanmean(registration["fn"], axis=0),
    color=OKABE_ITO["blue"],
    linewidth=2.4,
)
axes[1].set_title("Registered curves")

axes[2].plot(
```

```
    movement_grid,
    registration["gam"].T,
    color=OKABE_ITO["black"],
    alpha=0.12,
    linewidth=0.7,
)
axes[2].plot(
    movement_grid,
    movement_grid,
    color=OKABE_ITO["blue"],
    linewidth=2.4,
)
axes[2].set_title("Warping functions")

for ax in axes:
    ax.set_xlabel("normalized movement time")
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

axes[0].set_ylabel("linear velocity (cm/s)")
axes[1].set_ylabel("linear velocity (cm/s)")
axes[2].set_ylabel("warped time")

fig.tight_layout()
```



A2. FDA lag

`fda_lag` is the signed area between a trial's warping function and the identity line. It can be analyzed with the behavioral, geometric, and kinematic measures in the scalar table.

In [21]:

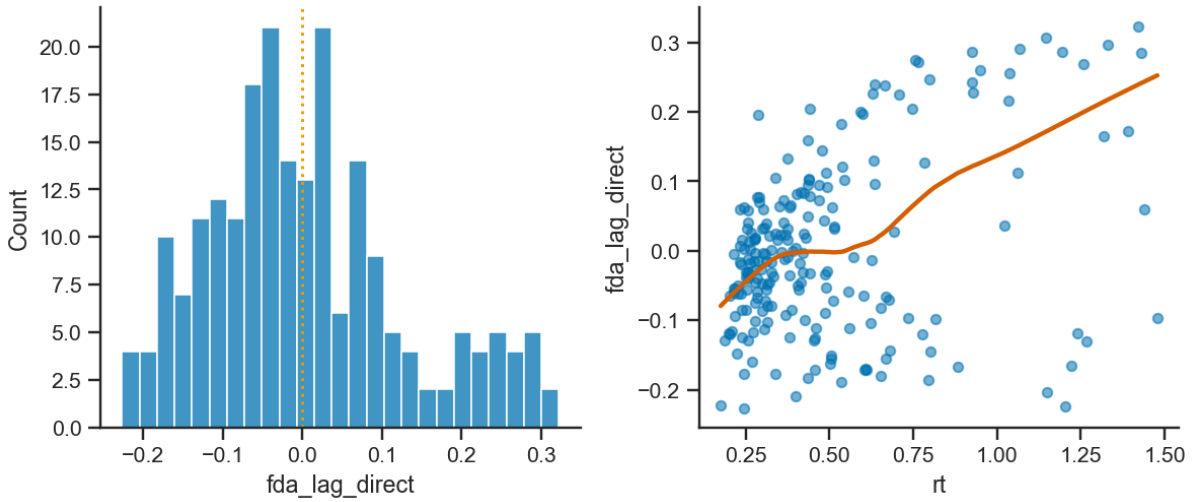
```
fda_detail = pd.DataFrame({
    "idx": movement_meta["idx"].astype(int).to_numpy(),
    "fda_lag_direct": registration["lag"],
    "fda_amp1_direct": registration["amp"][:, 0],
})

fda_with_scalars = measure_table.merge(
    fda_detail,
    on="idx",
```

```
    how="left",
)

lag_variables = [
    name for name in (
        "fda_lag",
        "fda_lag_direct",
        rt_measure,
        "sampling",
        "peak_lin_vel",
        "dev_unchosen_norm",
    )
    if name in fda_with_scalars.columns
]

display(
    fda_with_scalars[lag_variables]
    .corr(method="spearman")
    .round(2)
)
```

	fda_lag	fda_lag_direct	rt	sampling	peak_lin_vel	dev_
fda_lag	1.00	1.00	0.25	-0.20	-0.05	
fda_lag_direct	1.00	1.00	0.25	-0.20	-0.05	
rt	0.25	0.25	1.00	-0.34	0.06	
sampling	-0.20	-0.20	-0.34	1.00	0.01	
peak_lin_vel	-0.05	-0.05	0.06	0.01	1.00	
dev_unchosen_norm	0.50	0.50	0.62	-0.44	0.11	

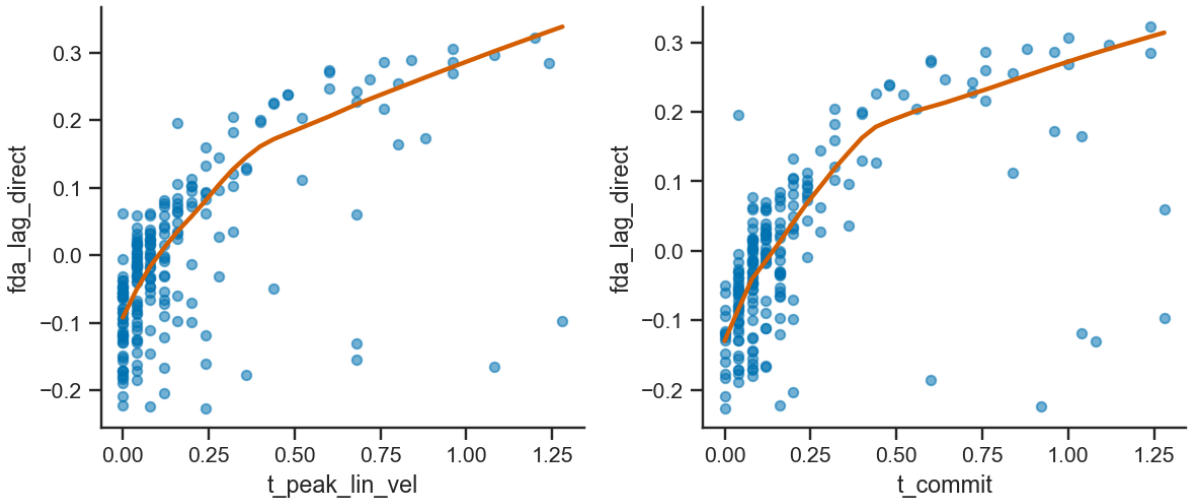
In [22]:
```
fig, axes = plt.subplots(1, 2, figsize=(8.67, 3.8), dpi=140)

sns.histplot(
    data=fda_with_scalars,
    x="fda_lag_direct",
    bins=25,
    ax=axes[0],
    color=OKABE_ITO["blue"],
)
axes[0].axvline(0, color=OKABE_ITO["orange"], linestyle=":")

sns.regplot(
    data=fda_with_scalars,
    x=rt_measure,
    y="fda_lag_direct",
    lowess=True,
    ax=axes[1],
```

```
    scatter_kws={"color": OKABE_ITO["blue"], "s": 24, "alpha": 0.55},
    line_kws={"color": OKABE_ITO["vermillion"]},
)

for ax in axes:
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

fig.tight_layout()
```



In [23]:

```
fig, axes = plt.subplots(1, 2, figsize=(8.67, 3.8), dpi=140)

sns.regplot(
    data=fda_with_scalars,
    x="t_peak_lin_vel",
    y="fda_lag_direct",
    lowess=True,
    ax=axes[0],
    scatter_kws={"color": OKABE_ITO["blue"], "s": 24, "alpha": 0.55},
    line_kws={"color": OKABE_ITO["vermillion"]},
)

sns.regplot(
    data=fda_with_scalars,
    x="t_commit",
    y="fda_lag_direct",
    lowess=True,
    ax=axes[1],
    scatter_kws={"color": OKABE_ITO["blue"], "s": 24, "alpha": 0.55},
    line_kws={"color": OKABE_ITO["vermillion"]},
)

for ax in axes:
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

fig.tight_layout()
```



A3. Functional PCA: amplitude scores and shape modes

The curves are analyzed after registration. Each score is a scalar amplitude along one functional mode. The positive and negative mode curves show the associated change in curve shape. Component signs are arbitrary.

Suppression of runtime issues are required due to the underlying skfda package, and is not due to any issues in ERPA.

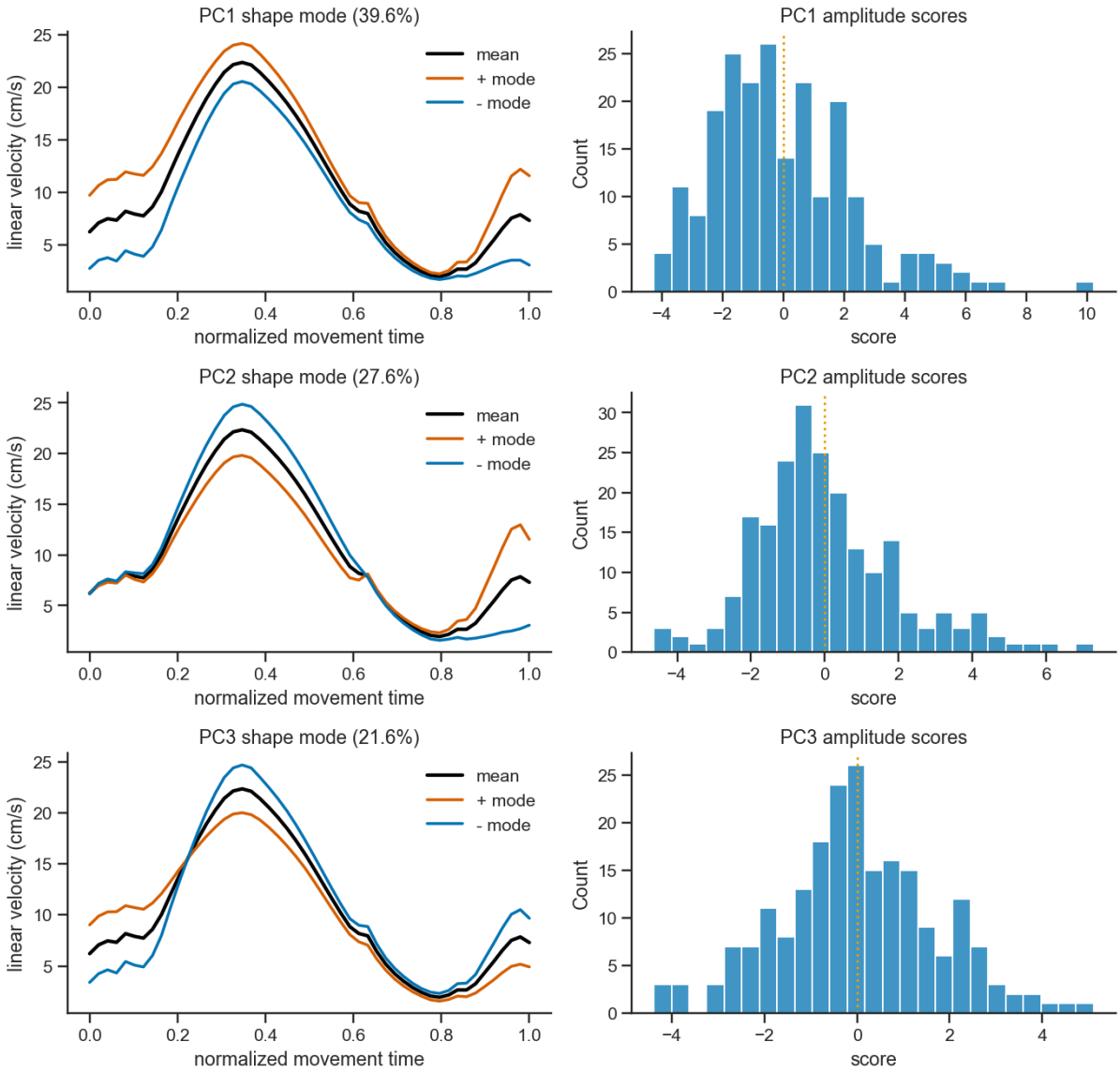
In [24]:
```
import warnings

# This block isolates and suppresses the warning
with warnings.catch_warnings():
    warnings.simplefilter("ignore", category=RuntimeWarning)

    n_fpca = min(
        FDA_N_COMPONENTS,
        registration["fn"].shape[0] - 1,
        registration["fn"].shape[1] - 1,
    )

    fd, fpca, fpca_scores, fpca_variance = run_fpca(
        movement_grid,
        registration["fn"],
        n_components=n_fpca,
    )

    mean_registered = np.nanmean(registration["fn"], axis=0)
    components = fpca.components_.data_matrix[:n_fpca, :, 0]
    component_sd = np.sqrt(fpca.explained_variance_[:n_fpca])

    mode_curves = [
        (
```

```
            mean_registered + component_sd[j] * components[j],
            mean_registered - component_sd[j] * components[j],
        )
        for j in range(n_fpca)
    ]
```

In [25]:

```
fig, axes = plt.subplots(
    n_fpca,
    2,
    figsize=(10, 3.2 * n_fpca),
    dpi=140,
    squeeze=False,
)

for j in range(n_fpca):
    positive, negative = mode_curves[j]

    axes[j, 0].plot(
        movement_grid,
        mean_registered,
        color=OKABE_ITO["black"],
        linewidth=2.2,
        label="mean",
    )
    axes[j, 0].plot(
        movement_grid,
        positive,
        color=OKABE_ITO["vermillion"],
        linewidth=1.8,
        label="+ mode",
    )
    axes[j, 0].plot(
        movement_grid,
        negative,
        color=OKABE_ITO["blue"],
        linewidth=1.8,
        label="- mode",
    )
    axes[j, 0].set_title(
        f"PC{j + 1} shape mode ({fpca_variance[j] * 100:.1f}%)"
    )
    axes[j, 0].set_xlabel("normalized movement time")
    axes[j, 0].set_ylabel("linear velocity (cm/s)")
    axes[j, 0].legend(frameon=False)

    sns.histplot(
        fpca_scores[:, j],
        bins=25,
        ax=axes[j, 1],
        color=OKABE_ITO["blue"],
    )
    axes[j, 1].axvline(0, color=OKABE_ITO["orange"], linestyle=":")
```

```
    axes[j, 1].set_title(f"PC{j + 1} amplitude scores")
    axes[j, 1].set_xlabel("score")

    for ax in axes[j]:
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)

fig.tight_layout()
```



In [26]:
```
fpca_score_table = movement_meta[["idx"]].copy()

for j in range(n_fpca):
    fpca_score_table[f"fda_shape_score{j + 1}"] = fpca_scores[:, j]

fpca_with_scalars = measure_table.merge(
    fpca_score_table,
    on="idx",
    how="left",
)
```

```
display(
    fpca_with_scalars.filter(
        regex=r"^(idx|rt|RT|sampling|peak_lin_vel|fda_)"
    ).head()
)
```

	idx	sampling	rt	peak_lin_vel	fda_lag	fda_shape_score1	fda_shape_s
0	0	0.038	0.796	20.229069	-0.185138	-1.051360	2.9
1	1	0.146	0.256	18.067765	0.041098	-1.092123	0.1
2	2	0.116	0.280	25.092943	-0.074733	0.418130	-1.3
3	3	0.164	1.222	23.906687	-0.165305	1.564946	4.2
4	4	0.144	0.214	22.725911	-0.004902	0.759559	-1.6

A4. Functional curves and time-series classification

Raw curves retain phase and amplitude differences. Registered curves reduce phase variability and emphasize amplitude and shape. Either representation can be supplied to ROCKET-family classifiers or shapelet methods, depending on the scientific question. Preprocessing and feature extraction should be fit within each cross-validation fold.

In [27]:
```
X_raw = movement_velocity[:, np.newaxis, :]
X_registered = registration["fn"][:, np.newaxis, :]

print("Raw functional array:", X_raw.shape)
print("Registered functional array:", X_registered.shape)
```

```
Raw functional array: (213, 1, 50)
Registered functional array: (213, 1, 50)
```

In [28]:
```
np.savez_compressed(
    OUTPUT_DIR / "fda_movement_curves.npz",
    grid=movement_grid,
    raw=movement_velocity,
    registered=registration["fn"],
    warping=registration["gam"],
    lag=registration["lag"],
    fpca_scores=fpca_scores,
)

movement_meta.to_csv(
    OUTPUT_DIR / "fda_movement_metadata.csv",
    index=False,
```

```
)

fpca_score_table.to_csv(
    OUTPUT_DIR / "fda_fpca_scores.csv",
    index=False,
)
```